

PROJECT TESTING PHASE

ComicCraft – AI Comic Story Creator using Gemini Models

1. Introduction

The Project Testing Phase verifies that the ComicCraft application works correctly after development. The project document describes local deployment testing to verify the complete comic-generation workflow, including AI text generation, image creation, layout building, and PDF export. The application is tested through the local FastAPI server and its browser interface.

2. Objectives of Testing

- Verify that the FastAPI application starts correctly.
- Check that the ComicCraft homepage loads successfully.
- Verify that user inputs are accepted and processed.
- Test the AI-powered comic generation workflow.
- Verify image generation for comic panels.
- Check the comic layout and preview.
- Verify PDF generation and export.
- Test available FastAPI API endpoints.

3. Testing Environment

The project uses a Python virtual environment to isolate its dependencies. The required libraries are installed from the project's requirements, including FastAPI, Uvicorn, Google Generative AI, Hugging Face Diffusers, FPDF, and other required packages. The application is launched locally using Uvicorn.

The project document specifies the command **uvicorn app.main:app --reload** for starting the FastAPI development server.

4. Application and Input Testing

Testing the FastAPI Server

The first testing activity is to start the FastAPI server from the project root where `app/main.py` is located. Once the server is running, the application is accessed through the local browser address.

Local Application URL

The project document specifies **`http://127.0.0.1:8000`** as the local address for opening the ComicCraft homepage.

Testing User Input

The homepage contains a structured form that collects the information needed to generate a personalized comic. The following input fields are tested:

- Story Prompt – the main idea for the comic.
- Main Character Name – the name of the comic's main character.
- Setting – the location or environment of the story.
- Story Tone – the mood of the story.
- Art Style – the preferred visual style.

Expected Testing Result

The homepage should accept the required information and submit it to the FastAPI backend through the `/generate` route. The backend should then begin the comic-generation workflow.

Testing Form and JSON Input

The project supports both HTML form input and JSON-based API input. Form data is captured using FastAPI `Form(...)` parameters, while JSON input is validated through the Pydantic `PromptRequest` model.

5. AI Generation and Functional Testing

Testing the Comic Generation Workflow

The main functional test verifies that the different AI components work together to create a complete comic. The project document specifies the following sequence.

Step	Function Tested	Purpose
1	generate_outline()	Creates the structured five-panel comic outline.
2	generate_story()	Creates narration and character dialogue.
3	generate_image()	Creates comic-style illustrations for panels.
4	build_comic_layout()	Matches images and story content into a layout.
5	save_pdf()	Compiles the completed comic into a PDF.

Testing the AI Story Generation

Gemini Flash is used to generate the structured comic outline, while Gemini Pro creates detailed narration and character dialogue. Testing verifies that the generated content is passed correctly between these stages.

Testing Image Generation

The image generation function uses Stable Diffusion to create an illustration from each panel's image prompt. The generated image is saved to the static/panels directory, and its path is returned for use by the application.

6. API, Preview and Export Testing

FastAPI API Documentation Testing

FastAPI provides interactive API documentation at <http://127.0.0.1:8000/docs>. This interface allows developers to test the available API endpoints and verify their responses.

Endpoints Identified for Testing

- `/generate-comic/json` – tests JSON-based comic generation.
- `/generate` – tests form-based comic generation.
- `/export-success` – checks the export confirmation page.
- `/test-image` – tests image generation separately.

Comic Preview Testing

After generation, the Comic Preview Page displays the comic panels sequentially. Testing checks that each panel contains the expected visual and textual information.

- Panel number and title.
- AI-generated comic image.
- Scene description.
- Caption and narration.
- Image prompt reference.

PDF Export Testing

The Download Your Comic as PDF function is tested to verify that the complete comic is compiled into a PDF. The project document states that the exported PDF includes images, panel titles, scene descriptions, captions, and narration.

7. Local Deployment Verification and Conclusion

Local Deployment Testing

The final testing activity verifies that the complete ComicCraft workflow runs smoothly on the local system. The project document describes this as testing and verifying the local deployment before any future cloud deployment.

Complete Testing Flow

1. Start the FastAPI server using Uvicorn.
2. Open the ComicCraft homepage in a browser.
3. Enter the story prompt and comic preferences.
4. Submit the form and start comic generation.
5. Verify the generated outline, narration, dialogues, and images.
6. Verify the comic preview page and panel layout.
7. Test the PDF download function.
8. Verify the export-success page after downloading.
9. Use the FastAPI /docs interface to test supported API endpoints.

Expected Testing Outcome

The testing phase is intended to confirm that the complete comic-generation workflow—including AI text generation, image creation, layout building, and PDF export—runs smoothly in the local environment.

Conclusion

The Project Testing Phase verifies the major components of ComicCraft and their integration. Testing covers the FastAPI server, user input, AI generation functions, image generation, comic preview, API endpoints, PDF export, and local deployment. Successful verification provides a basis for using the completed application and considering future deployment enhancements.

Source: ComicCraft – AI Comic Story Creator using Gemini Models project document.